Reflection CSC360 Lecture 3

Understanding & Drawing a Centered Square

- Understand that the pixel is the fundamental unit of distance in 2D graphics.
- Understand how to locate a canvas's midpoint using its overall width and height.
- Understand how to derive all four vertices of a square from its center coordinate and side length.
- Understand the spatial difference between top-left, top-right, bottom-left, and bottom-right points.
- Practice drafting pseudocode to render a centered square on a screen.
- Understand standard Maven directory layouts for Java applications.
- Understand how JFrame functions as the main window container in Java Swing.
- Understand the distinction between static (class-wide) and instance (object-specific) variables.

Key Learning

The core focus of this session was positioning a square at the center of a 2D canvas using coordinate arithmetic. Screen coordinates differ from standard Cartesian planes: the origin (0,0) sits at the top-left, meaning X increases to the right while Y increases downward.

To place a square in the center, I first calculate the canvas midpoints:

- cx = W / 2
- cy = H / 2

For a square with side length L, the half-distance is L / 2. The four corners are calculated as:

- Top-Left: (cx - L/2, cy - L/2)

- Top-Right: (cx + L/2, cy - L/2)
- Bottom-Left: (cx - L/2, cy + L/2)
- Bottom-Right: (cx + L/2, cy + L/2)

I also studied standard Java Maven project organization, including pom.xml, src/main/java, and src/main/resources, learning how Maven structures code and dependencies.

Additionally, I explored JFrame as the primary top-level window container in Swing, and reviewed object-oriented concepts: static fields belong to the class and are shared globally, while instance fields store unique data for individual objects.

Reflection

This material clearly demonstrated how basic math directly powers 2D graphics programming. Offsetting coordinates from a central point showed how simple algebra is used to position elements accurately on screen.

Realizing that computer graphics invert the Y-axis compared to standard geometry was a key takeaway, as keeping this in mind is essential for avoiding positioning errors. Finally, learning about JFrame and Maven directory structures provided a solid foundation for how Java graphics projects are configured and built.